Book Recommender System (Collaborative-based Recommender System)

In [36]:

```
#Import the required libraries:
import pandas as pd
import numpy as np

from sklearn.metrics.pairwise import cosine_similarity

import warnings
warnings.filterwarnings("ignore")


#Import the datasets:
books = pd.read_csv('Books.csv')
users = pd.read_csv('Users.csv')
ratings = pd.read_csv('Ratings.csv')
```

Inspect all datasets:

In [37]:

```
books.head()
```

Out[37]:

	ISBN	Book-Title	Book-Author	Year-Of-Publication	Publisher	Image-URL-S	
0	0195153448	Classical Mythology	Mark P. O. Morford	2002	Oxford University Press	http://images.amazon.com/images/P/0195153448.0...	http://i
1	0002005018	Clara Callan	Richard Bruce Wright	2001	HarperFlamingo Canada	http://images.amazon.com/images/P/0002005018.0...	http://i
2	0060973129	Decision in Normandy	Carlo D'Este	1991	HarperPerennial	http://images.amazon.com/images/P/0060973129.0...	http://i
3	0374157065	Flu: The Story of the Great Influenza Pandemic...	Gina Bari Kolata	1999	Farrar Straus Giroux	http://images.amazon.com/images/P/0374157065.0...	http://i
4	0393045218	The Mummies of Urumchi	E. J. W. Barber	1999	W. W. Norton & Company	http://images.amazon.com/images/P/0393045218.0...	http://i

In [38]:

```
users.head()
```

Out[38]:

	User-ID	Location	Age
0	1	nyc, new york, usa	NaN
1	2	stockton, california, usa	18.0
2	3	moscow, yukon territory, russia	NaN
3	4	porto, v.n.gaia, portugal	17.0

	User-ID	Location	Age
4	5	farnborough, hants, united kingdom	NaN

In [39]:

```
ratings.head()
```

Out[39]:

	User-ID	ISBN	Book-Rating
0	276725	034545104X	0
1	276726	0155061224	5
2	276727	0446520802	0
3	276729	052165615X	3
4	276729	0521795028	6

In [40]:

```
#Remove the commas from the 'Location' attribute of users dataset:
users["Location"] = users["Location"].str.replace(",","")
```

In [41]:

```
users.head()
```

Out[41]:

	User-ID	Location	Age
0	1	nyc new york usa	NaN
1	2	stockton california usa	18.0
2	3	moscow yukon territory russia	NaN
3	4	porto v.n.gaia portugal	17.0
4	5	farnborough hants united kingdom	NaN

In [42]:

```
#Inspect for NULL/Empty Values in 'users' dataset for all columns:
users.isnull().sum()
```

Out[42]:

```
User-ID           0
Location          0
Age          110762
dtype: int64
```

In [43]:

```
users.shape
```

Out[43]:

```
(278858, 3)
```

In [44]:

```
#Assign age 0 to all users for whom we dont have the age data:
users.fillna(0,inplace=True)
```

In [45]:

```
users.isnull().sum()
```

Out[45]:

```
User-ID     0
```

```
Location    0
Age         0
dtype: int64
```

In [46]:

```
users.head()
```

Out[46]:

	User-ID	Location	Age
0	1	nyc new york usa	0.0
1	2	stockton california usa	18.0
2	3	moscow yukon territory russia	0.0
3	4	porto v.n.gaia portugal	17.0
4	5	farnborough hants united kingdom	0.0

Check for null values in all columns of all datasets now:

In [47]:

```
ratings.isnull().sum()
```

Out[47]:

```
User-ID         0
ISBN            0
Book-Rating     0
dtype: int64
```

In [48]:

```
books.isnull().sum()
```

Out[48]:

```
ISBN                 0
Book-Title           0
Book-Author          2
Year-Of-Publication  0
Publisher            2
Image-URL-S          0
Image-URL-M          0
Image-URL-L          3
dtype: int64
```

In [49]:

```
books.shape
```

Out[49]:

```
(271360, 8)
```

In [50]:

```
#Remove all null values in books:
books.dropna(inplace=True)
```

In [51]:

```
books.isnull().sum()
```

Out[51]:

```
ISBN                 0
Book-Title           0
Book-Author          0
Year-Of-Publication  0
```

```
Year-Of-Publication    0
Publisher              0
Image-URL-S            0
Image-URL-M            0
Image-URL-L            0
dtype: int64
```

In [52]:

```
users.isnull().sum()
```

Out[52]:

```
User-ID     0
Location    0
Age         0
dtype: int64
```

In [53]:

```
ratings.isnull().sum()
```

Out[53]:

```
User-ID        0
ISBN           0
Book-Rating    0
dtype: int64
```

In [ ]:

```

```

In [ ]:

```

```

In [54]:

```
#Merge ratings and books datasets based on ISBN:
ratings_with_name = ratings.merge(books,on='ISBN')
```

In [55]:

```
num_rating_df = ratings_with_name.groupby('Book-Title').count()['Book-Rating'].reset_ind
ex()
num_rating_df.rename(columns={'Book-Rating':'num_ratings'},inplace=True)
num_rating_df.head()
```

Out[55]:

	Book-Title	num_ratings
0	A Light in the Storm: The Civil War Diary of ...	4
1	Always Have Popsicles	1
2	Apple Magic (The Collector's series)	1
3	Ask Lily (Young Women of Faith: Lily Series, ...	1
4	Beyond IBM: Leadership Marketing and Finance ...	1

In [56]:

```
num_rating_df.dtypes
```

Out[56]:

```
Book-Title     object
num_ratings     int64
dtype: object
```

In [57]:

In [57]:

```
#Inspect the books ratings:
ratings_with_name.head()
```

Out[57]:

	User-ID	ISBN	Book-Rating	Book-Title	Book-Author	Year-Of-Publication	Publisher	Image-UI
0	276725	034545104X	0	Flesh Tones: A Novel	M. J. Rose	2002	Ballantine Books	http://images.amazon.com/images/P/034545104
1	276726	0155061224	5	Rites of Passage	Judith Rae	2001	Heinle	http://images.amazon.com/images/P/015506122
2	276727	0446520802	0	The Notebook	Nicholas Sparks	1996	Warner Books	http://images.amazon.com/images/P/044652080
3	276729	052165615X	3	Help!: Level 1	Philip Prowse	1999	Cambridge University Press	http://images.amazon.com/images/P/052165615
4	276729	0521795028	6	The Amsterdam Connection : Level 4 (Cambridge ...	Sue Leather	2001	Cambridge University Press	http://images.amazon.com/images/P/052179502

In [58]:

```
ratings_with_name.dtypes
```

Out[58]:

```
User-ID                int64
ISBN                  object
Book-Rating            int64
Book-Title            object
Book-Author           object
Year-Of-Publication   object
Publisher             object
Image-URL-S           object
Image-URL-M           object
Image-URL-L           object
dtype: object
```

In [59]:

```
#Now also include the average ratings:
avg_rating_df = ratings_with_name.groupby('Book-Title')['Book-Rating'].mean().reset_inde
x()
avg_rating_df.rename(columns={'Book-Rating':'avg_rating'},inplace=True)

popular_df = num_rating_df.merge(avg_rating_df,on="Book-Title")
popular_df.head()
```

Out[59]:

	Book-Title	num_ratings	avg_rating
0	A Light in the Storm: The Civil War Diary of ...	4	2.25
1	Always Have Popsicles	1	0.00
2	Apple Magic (The Collector's series)	1	0.00
3	Ask Lily (Young Women of Faith: Lily Series, ...	1	8.00
4	Beyond IBM: Leadership Marketing and Finance ...	1	0.00

In [60]:

```
#Include the popular books only(books which are rated by many people at least 250):
popular_df = popular_df[popular_df['num_ratings']>=250].sort_values('avg_rating',ascending=False).head(50)
#Remove all duplicate entries:
popular_df = popular_df.merge(books,on='Book-Title').drop_duplicates('Book-Title')[['Book-Title','Book-Author','Image-URL-M','num_ratings','avg_rating']]

popular_df.head()
```

Out[60]:

	Book-Title	Book-Author	Image-URL-M	num_ratings	avg_rating
0	Harry Potter and the Prisoner of Azkaban (Book 3)	J. K. Rowling	http://images.amazon.com/images/P/0439136350.0...	428	5.852804
3	Harry Potter and the Goblet of Fire (Book 4)	J. K. Rowling	http://images.amazon.com/images/P/0439139597.0...	387	5.824289
5	Harry Potter and the Sorcerer's Stone (Book 1)	J. K. Rowling	http://images.amazon.com/images/P/0590353403.0...	278	5.737410
9	Harry Potter and the Order of the Phoenix (Boo...	J. K. Rowling	http://images.amazon.com/images/P/043935806X.0...	347	5.501441
13	Harry Potter and the Chamber of Secrets (Book 2)	J. K. Rowling	http://images.amazon.com/images/P/0439064872.0...	556	5.183453

In [61]:

```
popular_df.avg_rating = popular_df.avg_rating.round().astype(int)
```

In [62]:

```
popular_df.head()
```

Out[62]:

	Book-Title	Book-Author	Image-URL-M	num_ratings	avg_rating
0	Harry Potter and the Prisoner of Azkaban (Book 3)	J. K. Rowling	http://images.amazon.com/images/P/0439136350.0...	428	6
3	Harry Potter and the Goblet of Fire (Book 4)	J. K. Rowling	http://images.amazon.com/images/P/0439139597.0...	387	6
5	Harry Potter and the Sorcerer's Stone (Book 1)	J. K. Rowling	http://images.amazon.com/images/P/0590353403.0...	278	6
9	Harry Potter and the Order of the Phoenix (Boo...	J. K. Rowling	http://images.amazon.com/images/P/043935806X.0...	347	6
13	Harry Potter and the Chamber of Secrets (Book 2)	J. K. Rowling	http://images.amazon.com/images/P/0439064872.0...	556	5

In [ ]:

```

```

In [ ]:

```

```

Main Recommender System:

In [63]:

```
x = ratings_with_name.groupby('User-ID').count()['Book-Rating'] > 200
```

In [64]:

```
learned_users = x[x].index
```

```
learned_users
```

Out[64]:

```
Index([   254,   2276,   2766,   2977,   3363,   4017,   4385,   6251,   6323,
         6543,
       ...
       271705, 273979, 274004, 274061, 274301, 274308, 275970, 277427, 277639,
       278418],
      dtype='int64', name='User-ID', length=811)
```

In [65]:

```
filtered_rating = ratings_with_name[ratings_with_name['User-ID'].isin(learned_users)]
y = filtered_rating.groupby('Book-Title').count()['Book-Rating']>=60

famous_books = y[y].index
```

In [66]:

```
famous_books
```

Out[66]:

```
Index(['1984', '1st to Die: A Novel', '2nd Chance', '4 Blondes',
       'A Bend in the Road', 'A Case of Need',
       'A Child Called \It\": One Child's Courage to Survive"',
       'A Civil Action', 'A Fine Balance',
       'A Heartbreaking Work of Staggering Genius',
       ...
       'White Oleander : A Novel',
       'White Oleander : A Novel (Oprah's Book Club)',
       'Wicked: The Life and Times of the Wicked Witch of the West',
       'Wild Animus', 'Winter Solstice', 'Wish You Well', 'Without Remorse',
       'Wuthering Heights',
       'Zen and the Art of Motorcycle Maintenance: An Inquiry into Values',
       '\O\" Is for Outlaw"'],
      dtype='object', name='Book-Title', length=500)
```

In [67]:

```
#Merge all the learned users with famous books:
final_ratings = filtered_rating[filtered_rating['Book-Title'].isin(famous_books)]
pt = final_ratings.pivot_table(index='Book-Title',columns='User-ID',values='Book-Rating'
)
pt.fillna(0,inplace=True)
pt.head(3)
```

Out[67]:

User-ID	254	2276	2766	2977	3363	4017	4385	6251	6323	6543	...	271705	273979	274004	274061	274301	274308	275
Book-Title																		
1984	9.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	10.0	0.0	0.0	0.0	0.0	0.0	
1st to Die: A Novel	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	9.0	...	0.0	0.0	0.0	0.0	0.0	0.0	
2nd Chance	0.0	10.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	

3 rows × 808 columns

In [68]:

```
similarity_scores = cosine_similarity(pt)
def recommend_books(book):
    index = np.where(pt.index==book)[0][0]
    similar_books = sorted(list(enumerate(similarity_scores[index])),key=lambda x:x[1],r
```

```
everse=True)[1:6]

    data = []
    for i in similar_books:
        #print(i)
        '''
        The above statement will give this kind-of output:
                (35, np.float64(0.2702651417103732))
                (377, np.float64(0.26396193711234966))
                (61, np.float64(0.2366937434740099))
                (449, np.float64(0.23299389358170397))
                (119, np.float64(0.22222254415660225))
        But we want recommendations in text form and not in the form of vectors. '''
        result = books[books['Book-Title']==pt.index[i[0]]].drop_duplicates('Book-Title'
)

        try:
            if not result.empty:
                row = result.iloc[0]

                data.append([
                    row['ISBN'],
                    row['Book-Title'],
                    row['Book-Author'],
                    row['Image-URL-S']
                ])
            else:
                print("Sorry, No recommendations found.")
        except Exception as e:
            print("Error occured while fetching recommendations.")


    return data

recommend_books("1984")
```

Out[68]:

```
[['0451526341',
  'Animal Farm',
  'George Orwell',
  'http://images.amazon.com/images/P/0451526341.01.THUMBZZZ.jpg'],
 ['0449212602',
  "The Handmaid's Tale",
  'Margaret Atwood',
  'http://images.amazon.com/images/P/0449212602.01.THUMBZZZ.jpg'],
 ['0060809833',
  'Brave New World',
  'Aldous Huxley',
  'http://images.amazon.com/images/P/0060809833.01.THUMBZZZ.jpg'],
 ['0345313860',
  'The Vampire Lestat (Vampire Chronicles, Book II)',
  'ANNE RICE',
  'http://images.amazon.com/images/P/0345313860.01.THUMBZZZ.jpg'],
 ['3257208626',
  'Fahrenheit 451',
  'Ray Bradbury',
  'http://images.amazon.com/images/P/3257208626.01.THUMBZZZ.jpg']]
```

Export the recommendation system:

In [69]:

```
import pickle

with open("books_recommender_system.pkl","wb") as f:
    pickle.dump(recommend_books,f)
```

Load the recommendation system:

In [70]:

In [70]:

```
with open("books_recommender_system.pkl","rb") as f:
    model = pickle.load(f)
print(model("1984"))
```

```
[['0451526341', 'Animal Farm', 'George Orwell', 'http://images.amazon.com/images/P/0451526341.01.THUMBZZZ.jpg'], ['0449212602', "The Handmaid's Tale", 'Margaret Atwood', 'http://images.amazon.com/images/P/0449212602.01.THUMBZZZ.jpg'], ['0060809833', 'Brave New World', 'Aldous Huxley', 'http://images.amazon.com/images/P/0060809833.01.THUMBZZZ.jpg'], ['0345313860', 'The Vampire Lestat (Vampire Chronicles, Book II)', 'ANNE RICE', 'http://images.amazon.com/images/P/0345313860.01.THUMBZZZ.jpg'], ['3257208626', 'Fahrenheit 451', 'Ray Bradbury', 'http://images.amazon.com/images/P/3257208626.01.THUMBZZZ.jpg']]
```

In [ ]:

```

```

Project by:
Name: Om Satyawan Pathak
Email: omsatyawanpathakgit@gmail.com
LinkedIn: www.linkedin.com/in/om-satyawan-pathak-029b02368

In [ ]:

```

```

In [ ]:

```

```